

SIMATS ENGINEERING

CO 4: ASSESSMENT TOOL 3: Comparative Analysis

COURSE NAME: Operating Systems

COURSE CODE: CSA0408

COURSE OUTCOME COVERED: *co4: Optimize various file allocation strategies and disk scheduling algorithms to identify optimal methods for enhancing system performance.*

Assessment Tool Weightage: 30%

Course Outcome 1 Weightage: 20%

Dr.J.Velmurugan / Professor
Dept. of CSE

QUESTIONS: 5

Total Marks:25

Annexure A

COMPARATIVE ANALYSIS

Code of Conduct:

*I D.NAVEEN KUMAR(192511401) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

D.NAVEEN KUMAR

Signature of the student:

Annexure A

COMPARATIVE ANALYSIS

S.No	Comparative Analysis Questions	Marks
1	Compare Contiguous and Linked file allocation methods in terms of access speed, memory utilization, and fragmentation. Identify which method is more suitable for sequential file access and justify your answer.	5
2	Differentiate between Linked and Indexed file allocation with respect to random access capability, storage overhead, and reliability. Explain which method is more efficient for large files.	5
3	Compare FCFS and SSTF disk scheduling algorithms based on seek time, fairness, and the possibility of starvation. Discuss which algorithm performs better under heavy disk load.	5
4	Analyze the differences between SCAN and C-SCAN disk scheduling algorithms in terms of head movement, response time, and uniformity of service. Which algorithm is more suitable for real-time systems and why?	5
5	Compare LOOK and C-LOOK disk scheduling algorithms with respect to efficiency, total head movement, and practical implementation. Explain which algorithm provides better performance and under what conditions.	5

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Total
Conceptual Understanding	30%					
Accuracy of Answers	25%					
Application of Knowledge	20%					
Time Management	15%					
Question Interpretation	10%					
Total						

1. Contiguous vs Linked File Allocation

Feature	Contiguous Allocation	Linked Allocation
Storage method	File blocks are stored consecutively	File blocks can be stored anywhere and are connected using pointers
Access speed	Very fast	Slower
Sequential access	Excellent	Good
Random/direct access	Excellent	Poor
Memory utilization	May waste space due to fragmentation	Better disk-space utilization
External fragmentation	Yes	No external fragmentation
Pointer overhead	No pointer required for every block	Pointer required in every block
File growth	Difficult if adjacent free space is unavailable	Easy to increase file size
Reliability	High	Lower because a damaged pointer may break the file chain

Which is better for sequential access?

Contiguous allocation is generally more suitable for sequential file access when performance is the main concern.

Because all blocks are physically adjacent, the disk head can read them continuously with very little head movement.

For example:

Block 20 → 21 → 22 → 23 → 24

In linked allocation, blocks may be:

20 → 75 → 36 → 102 → 48

Therefore, the disk head may need to move to several different locations.

Conclusion: Contiguous allocation provides faster sequential access, while linked allocation provides better flexibility and disk-space utilization.

2. Linked vs Indexed File Allocation

Feature	Linked Allocation	Indexed Allocation
Block organization	Blocks connected using pointers	Index block stores addresses of file blocks
Sequential access	Good	Good
Random access	Poor	Excellent
Storage overhead	Pointer stored in every data block	Separate index block required
File growth	Easy	Easy within index capacity
Reliability	Lower	Better
Pointer failure	Can break remaining file chain	Failure of index block can affect whole file
Large file handling	Less efficient	More efficient
Fragmentation	No external fragmentation	No external fragmentation

Random Access

Suppose a file contains:

B1 → B2 → B3 → B4 → B5

In linked allocation, to access B5, the system generally follows:

B1 → B2 → B3 → B4 → B5

Therefore random access is inefficient.

In indexed allocation:

Index Block

0 → B1

1 → B2

2 → B3

3 → B4

4 → B5

The address of B5 can be obtained directly from the index.

Which is more efficient for large files?

Indexed allocation is generally more efficient for large files, especially when both sequential and random access are required.

Reasons include:

- Direct access to any block
- No need to traverse previous blocks
- Faster searching inside large files
- Supports multilevel indexing for very large files

However, indexed allocation requires additional storage for index blocks.

Conclusion: Linked allocation is simple and good for sequential access, whereas indexed allocation is better for large files requiring fast random access.

3. FCFS vs SSTF Disk Scheduling

FCFS – First Come First Serve

Requests are processed in the order in which they arrive.

Example:

Requests:

98, 183, 37, 122

FCFS serves:

98 → 183 → 37 → 122

SSTF – Shortest Seek Time First

The request closest to the current disk-head position is processed first.

Feature	FCFS	SSTF
Selection method	Arrival order	Nearest request first
Average seek time	Higher	Lower
Head movement	Usually high	Usually lower
Performance	Moderate	Better
Fairness	Very fair	Less fair
Starvation	No starvation	Possible
Implementation	Simple	More complex
Heavy disk load	Lower performance	Better average performance

Seek Time

SSTF normally provides lower average seek time because it chooses the closest request.

Suppose the head is at:

50

Requests:

10, 45, 55, 100

SSTF chooses:

50 → 45 → 55 → 10 → 100

rather than simply processing requests according to arrival order.

Fairness

FCFS provides better fairness because every request is eventually processed according to arrival order.

SSTF may continuously choose nearby requests and delay distant requests.

Starvation

FCFS:

No starvation

SSTF:

Starvation is possible.

For example, if the head is around track 50 and requests continuously arrive around tracks 40–60, a request at track 190 may wait for a long time.

Under Heavy Disk Load

SSTF normally performs better than FCFS under heavy disk load in terms of average seek time and throughput.

However, starvation is its major disadvantage.

Conclusion: SSTF provides better performance, while FCFS provides better fairness and simplicity.

4. SCAN vs C-SCAN Disk Scheduling

SCAN

SCAN is also called the **elevator algorithm**.

The disk head moves in one direction while servicing requests.

Example:

0 -----> 199

Head → → → → →
Services requests

After reaching the end:

← ← ← ← ←
Services requests again

Thus, requests are serviced in **both directions**.

C-SCAN

C-SCAN means **Circular SCAN**.

The disk head services requests in only one direction.

After reaching one end, it returns to the beginning without servicing requests during the return.

0 → → → → → 199

|

↓

0 ----- 199

|

Again →

Feature	SCAN	C-SCAN
Full form	Scan	Circular Scan

Feature	SCAN	C-SCAN
Servicing direction	Both directions	One direction
Head movement	Back and forth	Circular
Average seek time	Usually good	Good
Response time	Can vary depending on location	More uniform
Fairness	Good	Better uniformity
Waiting time	Middle tracks can get some advantage	More evenly distributed
Return trip	Processes requests	Does not normally process requests

Head Movement

SCAN can save movement because it processes requests while moving in both directions.

C-SCAN may involve more movement because the head returns from the final track to the starting area.

Response Time

SCAN may cause requests near the middle of the disk to receive better service than requests near the ends.

C-SCAN produces **more uniform waiting times** because requests are serviced in one consistent direction.

Which is better for real-time systems?

Between these two, **C-SCAN can be preferable when predictable and uniform response time is more important**, because waiting time is distributed more evenly among disk locations.

However, neither SCAN nor C-SCAN by itself guarantees hard real-time deadlines. True real-time systems may use deadline-aware disk scheduling.

Conclusion: SCAN is efficient in terms of movement, while C-SCAN gives more uniform and predictable service.

5. LOOK vs C-LOOK Disk Scheduling

LOOK and C-LOOK are improved versions of SCAN and C-SCAN.

The important difference is that the disk head **does not travel to the physical end of the disk unless a request exists there.**

LOOK

The head moves in one direction and services all requests until the **last pending request** in that direction.

Then it reverses.

Example requests:

20, 40, 60, 90, 120

Head = 50

Moving right:

50 → 60 → 90 → 120 → 40 → 20

The head stops at 120 rather than travelling to the maximum disk track.

C-LOOK

The head moves in only one servicing direction.

Example:

50 → 60 → 90 → 120

Then it jumps to:

20 → 40

It does not service requests while jumping from the highest request to the lowest request.

Feature	LOOK	C-LOOK
Based on	SCAN	C-SCAN
Servicing direction	Both directions	One direction
Goes to physical disk end	No	No
Stops at last request	Yes	Yes
Head movement	Usually lower	Can be slightly higher
Waiting time	Good	More uniform
Fairness	Good	More uniform fairness
Efficiency	Very high	Very high
Implementation	Relatively simple	Slightly more complex

Efficiency

Both LOOK and C-LOOK are more efficient than SCAN/C-SCAN in many workloads because unnecessary movement to the physical ends of the disk is removed.

For example, if the highest request is at track 150 but the disk contains 200 tracks:

SCAN may go:

150 → 199

LOOK stops at:

150

Therefore, unnecessary head movement is eliminated.

Which provides better performance?

There is no single winner for every workload.

LOOK usually provides better performance when the main objective is **minimizing total head movement and average seek time**, because it services requests while moving in both directions.

C-LOOK is better when **uniform waiting time and fairness across disk locations** are more important.

Conditions

Use **LOOK** when:

- Minimum head movement is important
- High throughput is required
- Requests are spread across the disk
- Uniform waiting time is less critical

Use **C-LOOK** when:

- More uniform response time is required
- Fair service to different disk locations is important
- Workload is heavy and continuously changing